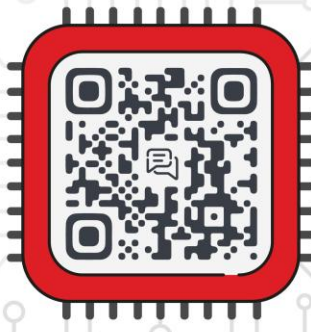




نظري

5



14



1680 sp

كلية الهندسة المعلوماتية

السنة الثالثة

## Advanced Pipelining

د. سيرا أستور

بنیان الحواسيب 2

RB Informatics; 10/11/2025

### Control Hazard أعطال التحكم:

تحصل عندما يحاول المعالج أن يتخذ قرار قبل أن يعرف نتيجة الشرط (مثل تعليمات القفز أو الفروع). سببها: التنفيذ خارج الترتيب (out-of-sequence execution)، حيث لا يعرف المعالج أي تعليمة يجب أن تجلب بعد التعليمة الحالية.

مثال: إذا كانت التعليمة الحالية هي Branch instruction، فهي ستحدد هل نكمل على التعليمة التالية (PC+4) أو نذهب إلى عنوان جديد.

#### المصطلحات التي سنستخدمها في هذه المحاضرة

Taken: الشرط محقق نريد القفز

Not Taken: الشرط غير محقق لا نريد القفز

#### الحالات:

Branch Taken: إذا تحقق الشرط، يتغير الـ PC من (PC+4) إلى العنوان العنوان (target) الجديد.

Branch Not Taken: إذا لم يتحقق الشرط، يبقى الـ PC كما هو (PC+4).

#### تذكرة:

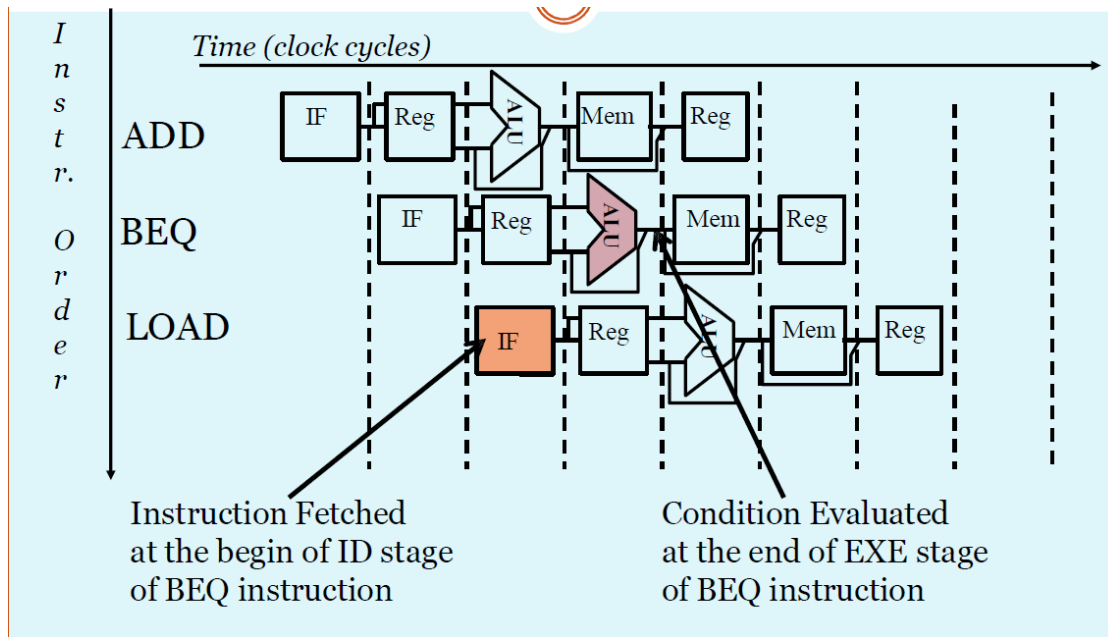
التعليمات الحقيقية بالبرانش هي إما مساواة أو عدم مساواة، أما ما تبقى من التعليمات هي تعليمات مزيفة

أبسط الطرق للتعامل مع الـ branch:

- تجميد الـ Pipeline الذي يحمل تعليمات بعد البرانش عن طريق stalls
- Flush the Pipeline لحذف جميع التعليمات بعد الـ branch إذا كان الشرط محقق (is taken)

مثال: BEQ (is taken)

يتحدد عنوان التعليمة التالية بعد التحقق من شرط الـ branch في EX stage ولكن التعليمة التالية قد تم جلبها في الـ Id Stage أي قبل أن تتغير الـ pc+4 للعنوان المستهدف الجديد مما قد يؤدي إلى control hazard الـ branch



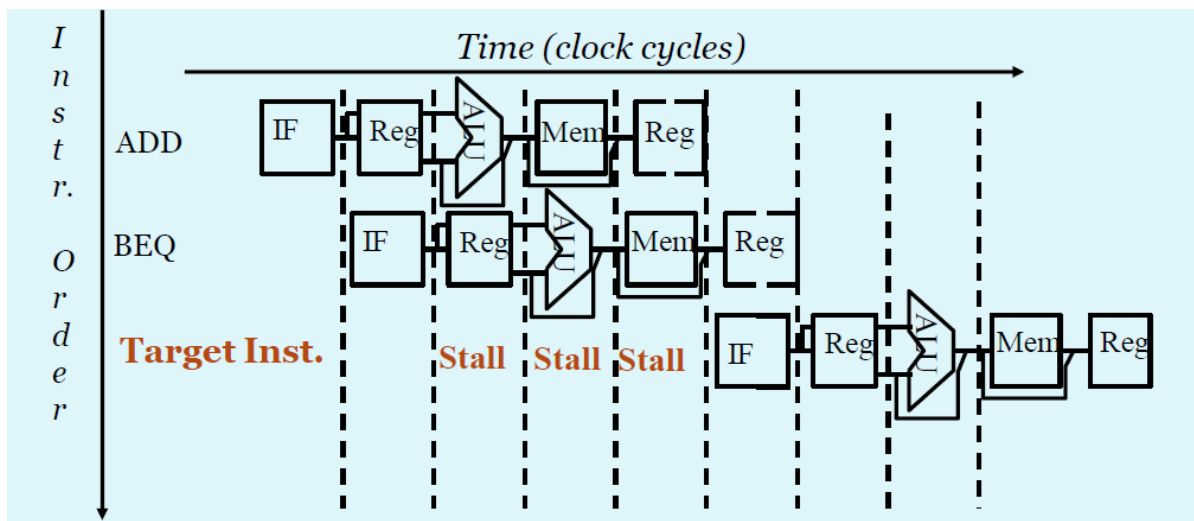
- المشكلة : أثناء التنفيذ، المعالج لا يعرف مسبقاً هل الفرع سيتحقق أم لا، وبالتالي قد يجلب تعليمات خاطئة.

### : Branch Hazards Solutions

- Stall
- Redo Fetch after branch : (بعد ما عملت Fetch برجع بعمل Fetch جديدة)
- Delayed Branching : تأجيل تنفيذ تعليمات ال Branch (هام)
- Branch Prediction : التنبؤ (تخمين اتجاه الفرع Taken) أو (Not Taken) .
- Multiple Streams لا يوجد عندي تسلسل وحيد للتعليمات وانما أكثر من تسلسل

### : Control Hazard Solution

- Stalls



نلاحظ ان النتيجة تصبح متاحة بعد مرحلة EXE، و عنوان الهدف يظهر في المرحلة التالية. لذلك نحتاج إلى (Stalls) 3 أو على الأقل اثنتين (Stalls) :

#### ■ التعليمة الأولى (ADD) :

تبدأ بشكل طبيعي في المراحل :

- IF (جلب التعليمة)
- Reg (قراءة السجلات)
- ALU (تنفيذ العملية الحسابي)

#### ■ التعليمة الثانية (BEQ)

تُجلب وتدخل المراحل :

$IF \rightarrow Reg \rightarrow ALU$

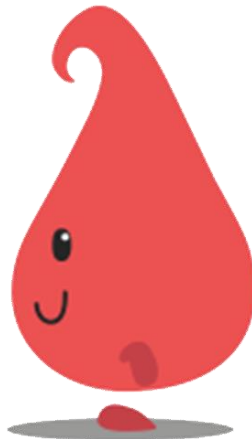
لكن نتيجة الشرط (هل الفرع سيتحقق أم لا) لا تظهر إلا في مرحلة (EXE) ALU. هذا يعني أن المعالج لا يعرف أي تعليمة يجب أن تُجلب بعد الـ BEQ .

#### ■ Target Instruction (تعليمة الهدف بعد الفرع):

- لا يمكن جلبها مباشرة لأن الـ PC لم يتحدد بعد.
  - لذلك يظهر في الجدول (Stall - Stall - Stall) ثلاث دورات توقف .
  - بعد أن يُعرف العنوان الجديد، تُجلب التعليمة الهدف وتكمل المراحل :
- $IF \rightarrow Reg \rightarrow ALU \rightarrow Mem \rightarrow Reg$

#### الخلاصة من الرسمة :

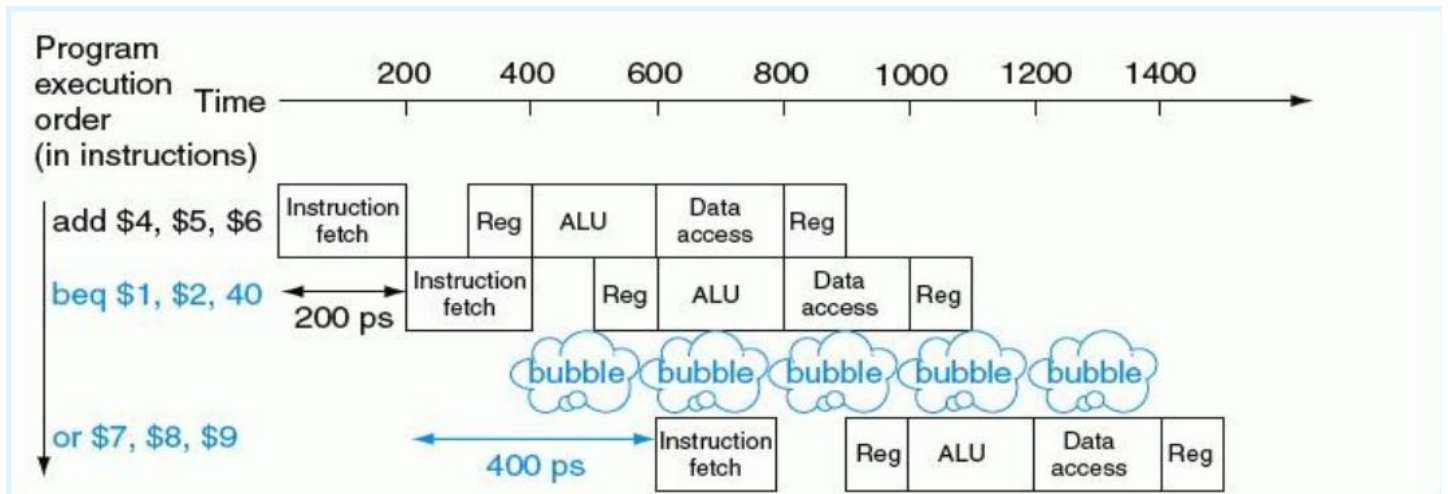
عند تعليمات الفرع مثل BEQ، المعالج يحتاج أن ينتظر حتى يتم تقييم الشرط. هذا الانتظار يؤدي إلى 3 دورات توقف (Stalls) قبل أن تُجلب التعليمة الصحيحة. الهدف من الرسمة هو توضيح أن الحل الأول لمشكلة الـ Control Hazard هو Stalling، أي إيقاف خط التنفيذ مؤقتاً حتى يُعرف العنوان الصحيح. نلاحظ أن كل تعليمة Branch تحتاج إلى 3 دورات توقف (Stalls) وهذا مكلف جداً لذلك سنسعى إلى تخفيفها (Reducing number of Stall)



## Reducing number of Stall (one stall is inevitable)

الهدف هو تقليل عدد دورات التوقف (Stalls) ، لكن وجود توقف واحد على الأقل أمر لا يمكن تجنبه.  
Extra hardware in ID phase (إضافة عتاد إضافي في مرحلة Decode) :

- Additional ALU to compute branch address  
وحدة حساب إضافية (ALU) لحساب عنوان الفرع مباشرة.
- Comparator to generate zero signal  
وحدة مقارنة لتوليد إشارة الصفر (Zero) لتحديد تحقق الشرط.
- Hazard detection unit writes the branch address in PC  
وحدة كشف التعارض تقوم بكتابة عنوان الفرع في عداد البرنامج (PC) .



عند تنفيذ تعليمة فرع مثل BEQ، المعالج يحتاج أن يعرف:

1. عنوان الفرع (Branch Address)
2. نتيجة الشرط من المقارنة (Zero signal)

في التصميم العادي: هذه المعلومات لا تتوفر إلا بعد مرحلة EXE، مما يسبب عدة دورات توقف (Stalls) .

في الرسمة: تمت إضافة وحدات جديدة داخل مرحلة ID :

- ALU إضافية لحساب عنوان الفرع مباشرة.
- Comparator لمقارنة القيم وتوليد نتيجة المقارنة بسرعة.
- Hazard Detection Unit تكتب عنوان الفرع في الـ PC مباشرة

النتيجة :

بدلاً من الانتظار حتى مرحلة التنفيذ (EXE) ، يتم تحديد عنوان الفرع ونتيجة الشرط في مرحلة ID هذا يقلل عدد دورات التوقف من ثلاث دورات الى دورة واحدة فقط (لكن لا يمكن التخلص من التوقف كلياً ، فواحدة تبقى حتمية).

الخلاصة من الرسمة :

الرسمة تبين أن الحل هو إضافة عتاد إضافي في مرحلة ID لتسريع معالجة تعليمات الفرع.

**النتيجة:** تقليل عدد الـ Stalls من ثلاثة إلى واحد فقط، مما يحسن الأداء ويقلل الهدر في خط التنفيذ.

## Redo Fetch after Branch

(وقف غير يلي جبتها روح جيب جديدة )

- عند كشف وجود فرع أثناء مرحلة Instruction Decode / Register Read ، تعتبر دورة الجلب التالية بمثابة Stall (توقف)، إذا افترضنا أن الفرع سيتم أخذه.
- التعليمات التي تُجلب في هذه الدورة لا تقوم بعمل مفيد، ويتم تجاهلها.
- بعد تحديد عنوان الفرع الصحيح، يقوم المعالج بإعادة الجلب (Re-fetch) للتعليمات التالية للفرع، مما يعطي التعليمات الصحيحة.
- إذا لم يتم أخذ الفرع (Not Taken) فلا حاجة لإعادة الجلب، لأن التعليمات الأصلية صحيحة.

⚡ (Impact) :

في حالة الفرع غير مأخوذ، التأخير يكون بمقدار دورة ساعة واحدة فقط لكل تعليمات فرع. هذا الحل أفضل من الحل الأول (Stalls) لأنه يقلل عدد الدورات الضائعة من ثلاث إلى دورة واحدة فقط.

الخلاصة المبسطة:

الحل الثاني يعتمد على أن المعالج يجلب التعليمات مرتين: الأولى قد تكون خاطئة إذا الفرع اتنفذ، والثانية بعد معرفة العنوان الصحيح. النتيجة: بدل ما نضيع 3 دورات، منضيع دورة وحدة بس إذا الفرع ما اتنفذ.



## Delayed Branch

الفكرة الأساسية هي وضع تعليمة :

- ليس لها علاقة بالبرانش
  - و تنفذ دائما
  - وهي هنا لدينا تعليمة add داخل البرانش
- فبذلك تتحول ال latency من 1 إلى 0

➤ مثال :

1. الوضع التقليدي (Stall on branch) :

- add \$4, \$5, \$6*  
*beq \$1, \$2, skip*  
*next instruction*  
*skip: or \$7, \$8, \$9*
- هنا المعالج يوقف (Stall) حتى يعرف نتيجة الفرع، وبالتالي نخسر دورة أو أكثر.

2. الوضع المحسن (Delayed branch) :

*beq \$1, \$2, skip*  
*add \$4, \$5, \$6*  
*next instruction*  
 ...  
*skip: or \$7, \$8, \$9*

- هنا التعليمة *add \$4, \$5, \$6* تنفذ بشكل مؤكد في Delay Slot ، سواء الفرع مأخوذ أو لا.

➤ (Impact)

- إذا قدرنا نلاقي تعليمة مناسبة نخططها في Delay Slot ، سيكون التأخير = 0 (Latency) دورة ساعة لكل تعليمة فرع.
- عملياً، هذا ممكن يتحقق بحوالي 50% من الحالات، لأنه لا يوجد لدينا دائما تعليمة مناسبة.

الخلاصة

الحل الثالث هو Delayed Branching :المعالج ينفذ تعليمة واحدة بعد الفرع بشكل مؤكد، وبعدين يقرر إذا الفرع مأخوذ أو لا.  
 النتيجة: منستغل دورة كانت رح تضيع، وإذا قدرنا نملأ ال Delay Slot بتعليمة مفيدة، منلغي التأخير كلياً.

## Prediction (Branch Prediction)

Branch Prediction هي تقنية تستخدم لمعالجة مشكلة تعليمات القفز (Branch Instructions) داخل الـ Pipeline .

عند تنفيذ تعليمة Branch ، لا يعرف المعالج مباشرة ما إذا كان القفز سيحدث أم لا، لذلك يقوم بالتنبؤ باتجاه الفرع ويكمل تنفيذ التعليمات بناءً على هذا التنبؤ. في حال كان التنبؤ خاطئاً، يتم تنفيذ عملية Pipeline Flush ثم إعادة التنفيذ بالمسار الصحيح. يوجد نوعان أساسيان من التنبؤ:

- Predict Branch Taken
- Predict Branch Not Taken

### Implementation of Branch Prediction

يمكن تنفيذ Branch Prediction باستخدام:

- Register يحتوي على bit1
- Register يحتوي على bits2

وغالباً يتم تمثيل هذه الآليات باستخدام Finite State Machine (FSM) .

### أهمية Branch Prediction :

تكون Branch Prediction فعالة جداً في Program Loops، لأن تعليمات القفز داخل الحلقات تتكرر عدة مرات بنفس السلوك، مما يسمح للمعالج بتعلم النمط وتحسين الأداء.

### Static Branch Prediction

في هذا النوع يعتمد التنبؤ على السوفتوير (Compiler) .

أبسط أسلوب هو التنبؤ بأن الفرع Taken دائماً.

متوسط معدل الخطأ يساوي نسبة الفروع التي لا يتم أخذها (Not Taken) .

في برامج SPEC يبلغ معدل الخطأ تقريباً 34% .

### يمكن تحسين هذا الأسلوب عن طريق :

- جمع معلومات من تشغيلات سابقة للبرنامج (Profile Information) .
- ثم اتخاذ قرار التنبؤ وقت الترجمة (Compile Time) .



## Dynamic Branch Prediction

يعتمد هذا النوع على الهاردوير، وهو أكثر دقة من Static Prediction ولكنه أكثر تعقيداً.  
أبسط شكل له هو:

Branch Prediction Buffer

أو Branch History Table (BHT)

وهو عبارة عن ذاكرة صغيرة يتم الوصول لها باستخدام جزء من عنوان تعليمة ال Branch.

## Branch Prediction Buffer – Implementation

يمكن تنفيذ Branch Prediction Buffer:

- كذاكرة خاصة صغيرة تشبه ال Cache ويتم الوصول لها خلال مرحلة IF .
- أو بإضافة 2 Bits مع كل تعليمة داخل Instruction Cache

## أنواع Branch Prediction Buffer

## 1-bit Prediction Scheme

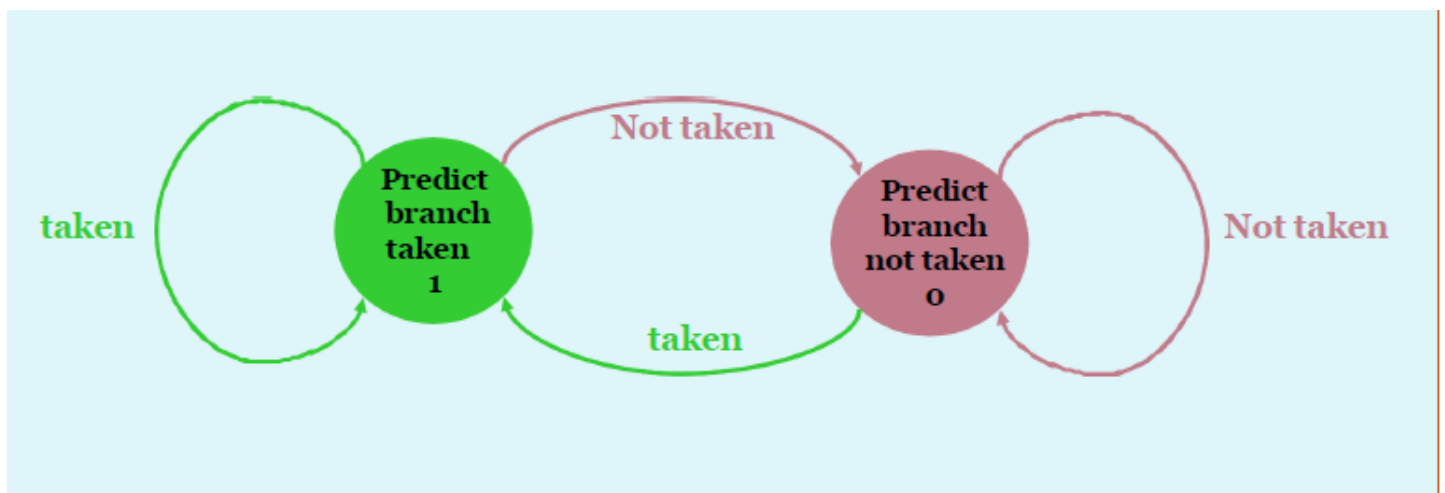
يحتوي على bit واحد يخزن نتيجة آخر مرة تم فيها تنفيذ الفرع:

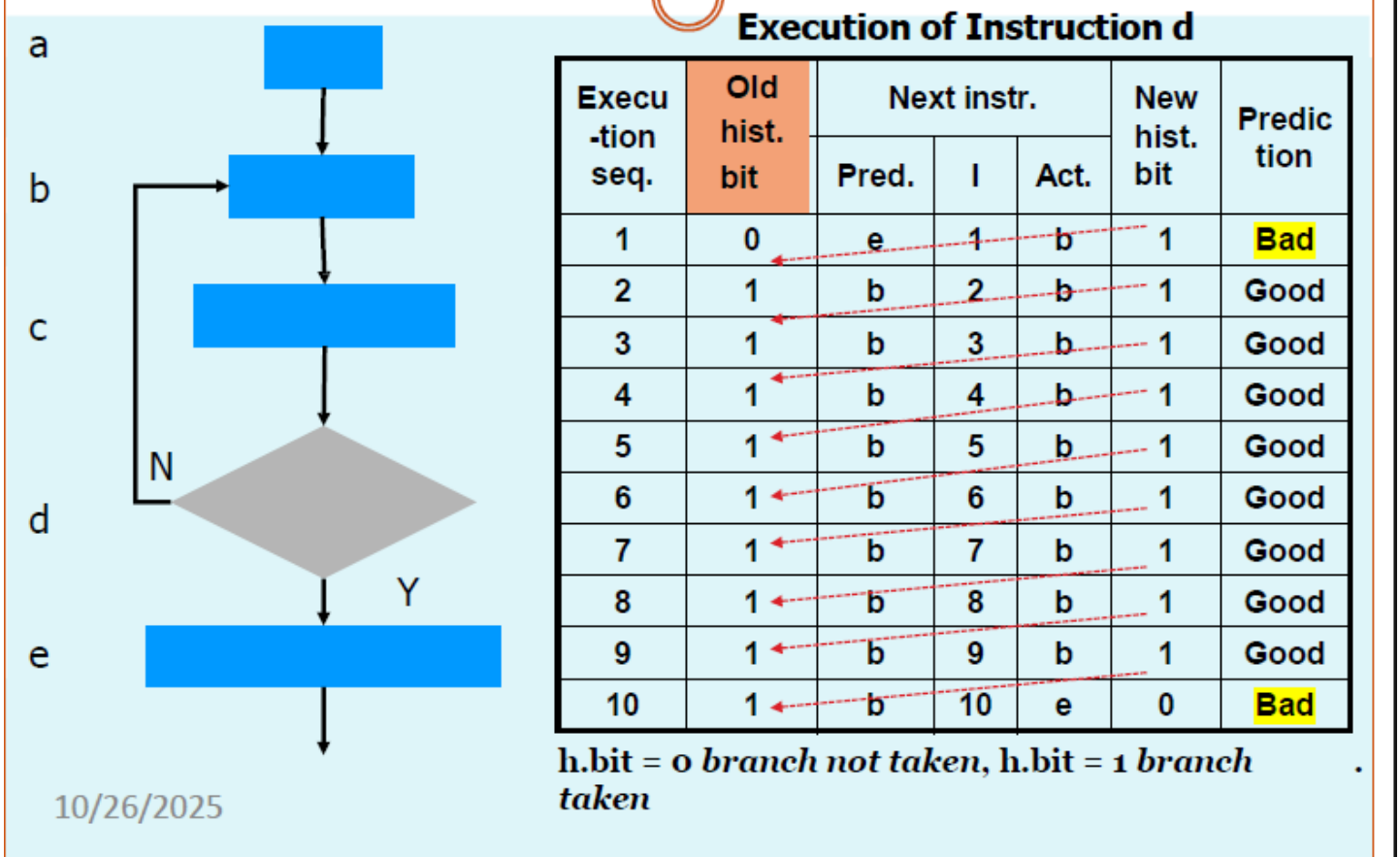
History bit = 1 → Branch Taken

History bit = 0 → Branch Not Taken

## 2-bit Prediction Scheme

لا يتم تغيير التنبؤ إلا بعد حدوث خطأين متتاليين، مما يزيد من ثبات ودقة التنبؤ.





المعالج يحاول يتوقع هل التعليمة الشرطية d ستعود للحلقة إلى b (branch taken) أو ستخرج منها إلى e (branch Not Taken)

يتم الاعتماد على آخر نتيجة فقط مخزنة في History Bit :

• 0 ← الفرع غير مأخوذ (Not Taken)

• 1 ← الفرع مأخوذ (Taken)

🔗 شرح الجدول:

- Old hist. bit : نتيجة آخر تنفيذ.
- Pred. : التوقع المبني على هذا البت.
- Act. : التنفيذ الحقيقي.
- New hist. bit : تحديث البت بعد التنفيذ.
- Prediction : هل التوقع كان صحيح (Good) أو خاطئ (Bad) .

🔗 ما الذي يحدث في الحلقة؟

- في أغلب مرات التنفيذ داخل الحلقة، الفرع يكون Taken ← التوقع صحيح (Good) .
- أول مرة يدخل فيها البرنامج الحلقة:
- التاريخ = 0 ← يتوقع الخروج ← لكن فعلياً يدخل ← توقع خاطئ.
- آخر مرة عند الخروج من الحلقة:
- التاريخ = 1 ← يتوقع الاستمرار ← لكن فعلياً يخرج ← توقع خاطئ.

الخلاصة المهمة:

متنبئ 1-bit جيد داخل الحلقة ، لكنه يفشل مرتين:

-عند الدخول الأول للحلقة

-عند الخروج النهائي منها

لهذا السبب تم تطوير 2-bit Branch Predictor لتحسين الدقة

مقارنة

1-bits Branch Prediction Buffer

في نظام 1-bit Predictor

-أول دخول للحلقة قد يسبب خطأ في التنبؤ

-أثناء تنفيذ الحلقة يكون التنبؤ صحيحاً

-عند الخروج من الحلقة يحدث خطأ آخر

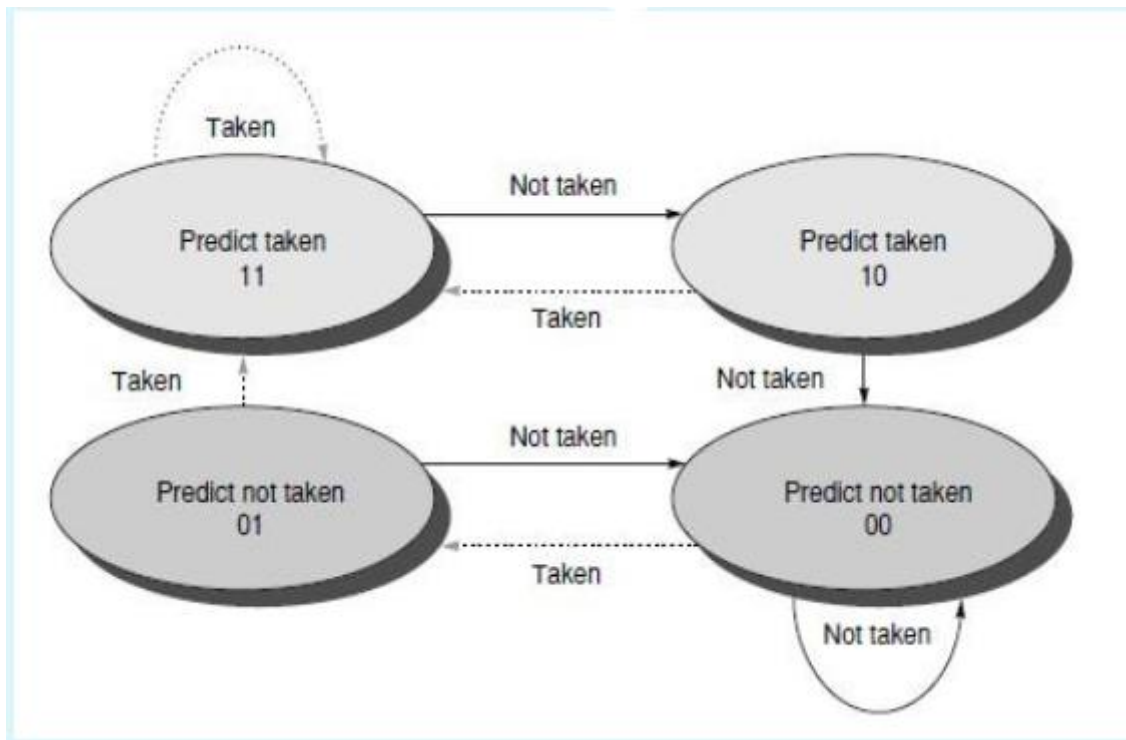
النتيجة:

2 أخطاء من أصل 10 تنبؤات ← Prediction Accuracy = 80%

المشكلة:

تغيير التنبؤ يحدث بسبب خطأ واحد فقط.

2-bits Branch Prediction Buffer



: 2-bit Branch Prediction

: 2-bit Predictor في نظام

لا يتم تغيير التنبؤ إلا بعد خطئين متتاليين

يتم استخدام FSM بأربع حالات:

Strongly Not Taken 00

Weakly Not Taken 01

Weakly Taken 10

Strongly Taken 11

هذا الأسلوب أكثر استقراراً وأعلى دقة.

الفكرة الأساسية :

بدل ما يعتمد المعالج على آخر نتيجة فقط،  
يستخدم عداد من 2 بت يعطي قرار أكثر استقراراً.

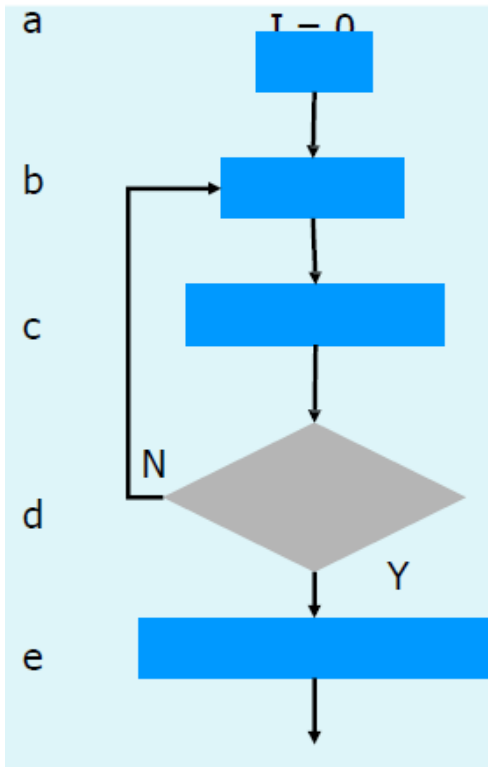
حالات التنبؤ:

Not Taken ← 01 أو 00

Taken ← 11 أو 10

التغيير في القرار يحتاج خطئين متتاليين، وليس خطأ واحد.





Execution of Instruction d

| Execution seq. | Old Pred. Buf | Next instr. |    |      | New pred. Buf | Prediction |
|----------------|---------------|-------------|----|------|---------------|------------|
|                |               | Pred.       | I  | Act. |               |            |
| 1              | 10            | b           | 1  | b    | 11            | Good       |
| 2              | 11            | b           | 2  | b    | 11            | Good       |
| 3              | 11            | b           | 3  | b    | 11            | Good       |
| 4              | 11            | b           | 4  | b    | 11            | Good       |
| 5              | 11            | b           | 5  | b    | 11            | Good       |
| 6              | 11            | b           | 6  | b    | 11            | Good       |
| 7              | 11            | b           | 7  | b    | 11            | Good       |
| 8              | 11            | b           | 8  | b    | 11            | Good       |
| 9              | 11            | b           | 9  | b    | 11            | Good       |
| 10             | 11            | b           | 10 | e    | 10            | Bad        |

## شرح الجدول

- Old Pred. Buf: قيمة العداد قبل التنفيذ.
- Pred. (التوقع): غالباً العودة إلى b
- Act.: التنفيذ الحقيقي.
- New Pred. Buf: تحديث العداد بعد التنفيذ.
- Prediction: هل التوقع صحيح أم لا.

## ما الذي يحدث داخل الحلقة؟

أثناء التكرار:

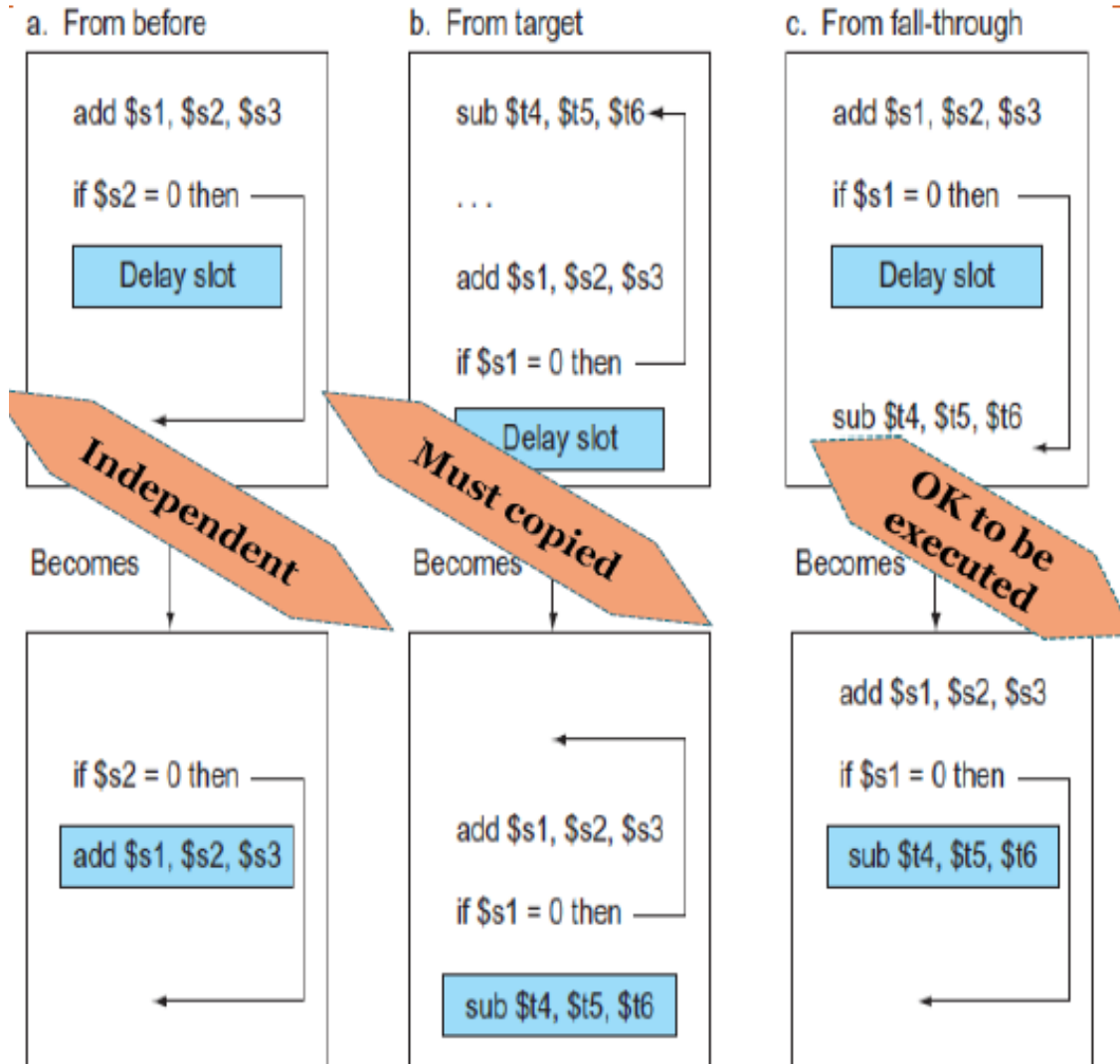
- العداد يبقى في الحالة 11 (Strongly Taken).
- التنبؤ دائماً صحيح

عند الخروج من الحلقة:

- المعالج يتوقع الاستمرار (Taken).
- لكن التنفيذ الحقيقي يخرج إلى e.
- يحصل خطأ واحد فقط.
- العداد ينخفض من 11 إلى 10 بدل ما ينعكس مباشرة.

الخلاصة المهمة : 2-bit predictor أدق بكثير من 1-bit.  
 داخل الحلقة:  
 - خطأ واحد فقط عند الخروج.  
 - لا يوجد خطأ عند الدخول.  
 لهذا السبب يُستخدم عملياً في المعالجات الحديثة.

## Branch Delay Slot



### استراتيجيات ملء الـ Delay Slot

#### (a) From before - Independent instruction

- نأخذ تعليمة قبل الـ branch ولا تعتمد على نتيجة الفرع ونضعها في الـ delay slot.

مثال:

```
add $s1, $s2, $s3
if $s2 == 0 then branch
```

بعد إعادة الترتيب:

```
if $s2 == 0 then branch
add $s1, $s2, $s3 # Delay Slot
```

## (b) From target – Must be copied

- نأخذ تعليمة من مسار القفز (Target)
- نضعها في delay slot
- لكن يجب نسخها أيضًا إلى مكانها الأصلي

مثال:

الفرع يقفز إلى:

```
sub $t4, $t5, $t6
```

بعد التعديل:

```
if $s1 == 0 then branch
add $s1, $s2, $s3 # Delay Slot
...
sub $t4, $t5, $t6 # Must be copied
```

## (c) From fall-through

- نأخذ تعليمة من المسار الطبيعي (عدم القفز)
- ونضعها في delay slot
- بشرط أنها لا تسبب خطأ إذا نُفذت أثناء القفز

مثال:

```
if $s1 == 0 then branch
sub $t4, $t5, $t6 # Delay Slot
```

